

Lecture 3 - Requirements

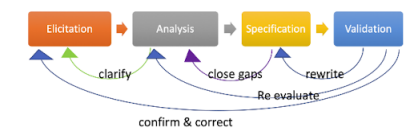
Requirements are a set of capabilities and constraints [IEEE]

1. A condition or capability needed by a user to solve a problem or achieve an objective
2. A condition or capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification, or other formally imposed documents
3. A documented representation fo a condition or capability as in (1) or (2)

Example of capability: system should allow a requester to specify pick-up and drop-off locations.

Example of constraint: pick-up and drop-off locations should be from a list of on-campus buildings / landmarks.

Requirements Development Phases



Elicitation: Discover requirements (through interview, workshop, document analysis, prototyping)

Analysis: Analyze, decompose, derive, understand, negotiate requirements, identify gaps

Specification: Written and illustrated requirements for comprehension, review, and use

Validation: Confirm correct set of requirements that will enable developers to build a solution

Sources of software requirements

- Documents
 - Descriptions of current or competing products
 - Standards or regulations
 - help desk problem reports
- Interviews
 - Focus groups
 - Use case workshops
 - Product champions as key customer rep
- Questionnaires and marketing surveys
 - Run a pilot first - difficult to ask clear questions
- Event-response tables
 - Identify external stimuli and describe system responses
- Observation - watch users do their jobs
 - Create workflow diagrams
 - "Day in the life" studies
 - Look at what information the user has when performing a particular task
- Prototyping
 - Useful for requirements, design, and implementation

FR vs NFR

FR describes something the system should do

- specify a behavior or function
- e.g. the passenger shall be able to print boarding passes for all flight segments for which she has checked in

NFR describes something not directly related to the functionality of the system

- May describe how well the system works
- Can include other attributes too (e.g. reliability, efficiency, usability)
- E.g. mean time between failure ≥ 9000 hrs

External vs Internal Quality Attributes

External:

- Observed when the software is executing
- Impacts user's experience of using the software
- Develops the user's perception of software quality
- E.g. Availability, performance, robustness, safety, security, reliability, deployability, compatibility, installability, usability, interoperability, integrity

Internal:

- Not directly observed when software is executing
- Perceived by the developers and maintainers
- Encompasses aspects of design that may impact external attributes
- E.g. Scalability, efficiency, modifiability, verifiability, portability, maintainability, reusability, testability